LLMProvider

LLMProvider

(repo: `llm_provider` · module `digital.vasic.llmprovider`)

Tagline: One interface, 43 providers — with circuit breakers, retries, and health baked in.

Summary: LLMProvider is a generic, reusable Go module that defines a unified `LLMProvider` interface plus the production resilience patterns around it — circuit breaker, health monitoring, retry with backoff, lazy loading — and ships 43 concrete provider implementations behind that one contract, with an OpenAI-compatible generic adapter and honest, no-hardcoded-fallback model discovery.

Short description: A reusable Go module exposing one `LLMProvider` interface (`Complete`, `CompleteStream`, `HealthCheck`, `GetCapabilities`, `ValidateConfig`) plus fault-tolerance primitives — circuit breaker, health monitor, jittered-backoff retry, lazy init — over 43 provider adapters and a generic OpenAI-compatible adapter. Thread-safe. (~40 words)

Long description: LLMProvider is the abstraction layer every LLM-consuming service needs but rarely builds well. It defines a single interface — `Complete`, `CompleteStream`, `HealthCheck`, `GetCapabilities`, `ValidateConfig` — so application code targets one contract regardless of backend, then ships the operational patterns that make provider calls survivable in production. A circuit breaker (closed/open/half-open) transparently wraps any provider, including its streaming channel, so a failing backend can’t cascade; a `CircuitBreakerManager` tracks them all. A configurable health monitor moves providers through healthy/degraded/unhealthy/unknown states on threshold-and-interval checks. Retry logic uses exponential backoff with jitter, status-aware decisions (retry 429/5xx, not 4xx or cancelled contexts), and clamped delays. A lazy pattern defers provider construction until first use — critical when 43 providers are registered but only a subset is used. The module ships 43 concrete provider packages plus a `generic` OpenAI-compatible adapter that implements the full interface against any `/v1/chat/completions` endpoint (Bearer auth, SSE streaming with `[DONE]` handling), so vendors without a

dedicated package still work. Credentials are resolved in exactly one place (`apikeys`, using an `ApiKey_<Provider>` convention) to prevent “hardcoded key passes tests, real key not wired” bugs. Model discovery is deliberately honest: it queries live provider APIs (with a TTL cache) and, per governance, the old hardcoded fallback tier was removed — when live discovery fails it returns nothing rather than a stale catalogue, so users are never handed model IDs they cannot invoke.

Why we built it: Naive LLM calls fail in production — providers rate-limit, degrade, or go down, and one bad backend can take a service with it. Model catalogues drift, and hardcoded lists hand callers IDs that no longer work. LLMProvider centralizes the interface, the resilience patterns, and honest discovery so every consumer inherits fault tolerance and truthfulness for free.

Why it’s a game-changer: It reduces “integrate an LLM provider” to implementing one interface (or pointing the generic adapter at an endpoint), then automatically wraps it in circuit breaking, health monitoring, and retry — turning provider reliability from per-project firefighting into a library default across 43 backends.

What’s innovative: - **One capability-aware interface** covering completion, streaming, health, capabilities, and config validation. - **Transparent circuit-breaker wrapping — including streams** (an empty stream counts as a failure), with deadlock-safe listener notification. - **43 provider packages + a generic OpenAI-compatible adapter**, so most providers are thin and unlisted vendors still work. - **Single credential authority (`apikeys`)** — one place reads `ApiKey_<Provider>` env vars, killing a class of “green tests, broken product” bugs. - **Honest model discovery (no hardcoded fallback)** — live provider APIs + TTL cache; on failure returns `nil` rather than a stale/false catalogue. - **Lazy init with `sync.Once`** — deferred construction so registering 43 providers is cheap. - **Anti-bluff, multi-locale Challenge stack** — real runner exercising circuit/health/retry across five locales with a paired mutation gate (normal → exit 0, mutate → exit 99).

Biggest technical challenges + how solved: - **Cascading provider failures.** Solved with a three-state circuit breaker that transparently wraps any provider (and its stream), managed centrally. - **Transient errors and rate limits.** Solved with status-aware exponential backoff + jitter (`min(InitialDelay·Multiplier^(n-1), MaxDelay) ± jitter`), retrying 429/500/502/503/504 and network errors, never cancelled contexts or other 4xx. - **Scaling to many registered-but-unused providers.** Solved with lazy initialization (`sync.Once`) so only used providers pay setup cost. - **Handing out invalid model IDs.** Solved by removing the hardcoded discovery fallback tier (per CONST-036) and returning nothing on live-discovery failure, with defensive copy-on-return to avoid caller mutation/races. -

Streaming + concurrency correctness. Solved by copying listeners before off-lock notification with a 5s timeout and unlocking before notifying on reset to avoid deadlock; all components built for concurrent use.

Tech stack (why + how): - **Go (1.25.3)** — the module, interface, resilience primitives, and adapters. - **`net/http` (stdlib)** — provider HTTP clients, the generic OpenAI-compatible adapter, and discovery calls. - **logrus** — structured logging in the circuit breaker and discovery. - **testify** — the test suite, including mutation-branch pinning. - **yaml.v3** — i18n bundles and config. - **`digital.vasic.models`** — shared `LLMRequest`/`LLMResponse`/`ProviderCapabilities` types (documented runtime dependency). - **First-party packages** — `circuit`, `health`, `retry`, `apikeys`, `discovery`, `providers/` (43 vendors + `generic`), `i18n`. - **`.env` + `~/api_keys.sh` (`ApiKey_<Provider>` convention)** — single-source credential resolution. - **Makefile race suite (`-race -p 1`) + Challenge runner** — anti-bluff verification (chaos, ddos, scaling, stress, live-discovery, no-suspend challenges).

Public links: - Product repo, PUBLIC: `github.com/HelixDevelopment/LLMProvider` - Also referenced in docs as `github.com/vasic-digital/llmprovider`. - Governance reference: `github.com/HelixDevelopment/HelixConstitution`. - **Note:** working origin remote is SSH (`git@github.com:HelixDevelopment/LLMProvider.git`); the GitHub repo is public. Provider endpoints referenced in code are public vendor URLs.

Suggested diagrams/illustrations (OpenDesign): 1. **Interface hub** — application → single `LLMProvider` interface → 43 adapters + the generic OpenAI-compatible adapter (fanning out to vendor endpoints). 2. **Circuit-breaker state machine** — closed → open → half-open, shown wrapping both `Complete` and the `CompleteStream` channel. 3. **Retry timeline** — exponential backoff + jitter with status-aware retry/skip decisions and context cancellation. 4. **Honest discovery** — live provider `/v1/models` + TTL cache; failure path returns nothing (old hardcoded tier crossed out), contrasted with the credential single-source (`apikeys`).

Site relevance: Both. Engineering-depth / AI-infrastructure feature on **vasic.digital** (pairs naturally with LLMsVerifier and HelixTranslate); reusable-module portfolio highlight on **milosvasic.ru**.

Priority tier: Helix-primary (LLM-infrastructure cluster — decoupled reusable module). Ranks after HelixTrack.

Source provenance: Local repo `/Volumes/T7/Projects/llm_provider` — `README.md`, `doc.go`, `docs/ARCHITECTURE.md`, `go.mod`, `provider.go`/`pkg/provider/provider.go`, `pkg/providers/` (43 packages incl. `generic/generic.go`), `pkg/apikeys/apikeys.go`, `pkg/discovery/discovery.go`, `pkg/circuit`/`pkg/health`/`pkg/retry`, `AGENTS.md`/`CLAUDE.md`/`QWEN.md`,

`challenges/`; cross-checked with harvested `_analysis/github-helix-others.md` (“40+ provider adapters”). Caution flags: license is inconsistent (doc.go says MIT; an Apache-2.0-style LICENSE file is present) — verify before publishing; `helix-deps.yaml` declares `deps: []` but docs state a hard dep on `digital.vasic.models` (manifest appears stale); discovery “Tier 2 (models.dev)” is a stub in the standalone module (planned, not active). LLMsVerifier is the upstream single source of truth for the canonical model catalogue; a HelixDevelopment “twin” of this module shares source parity but divergent git history. No references to HelixTranslate or helix_cluster. “Helix-Flow” appears only as an org name in governance boilerplate.